

Refresh Token Grant Type

- The Refresh Token grant type is used by clients to exchange a refresh token for an access token when the access token has expired.
- This allows clients to continue to have a valid access token without further interaction with the user.

Working Steps:

- 1) Copy authorization-code-type-server as **refresh-token-type-server**
- 2) Update **pom.xml**
- 3) Update **application.properties**
- 4) Update **MyAuthorizationServer.java**

A)Using Refresh Token with Authorization Code

Running Steps:

Step1: Send the Request to Authorizarion Server with the following URL to Get the User's Permission

A) Eendpoint - /oauth/authorize

B) Query Parameters

- client_id,
- redirect_uri,
- response_type,
- scope

http://localhost:12345/oauth/authorize?client_id=jlcappp&redirect_uri=http://localhost:5000/callback&response_type=code&scope=read_profile

http://localhost:5000/callback?code=mrYiWW

Step 2: Send the POST Request to Authorizarion Server with the following to get an access token using the previously received Authorization Code

A) Eendpoint - /oauth/ token

B) Query Parameters

- clientId,
- clientSecret



- code received
- redirect_uri,
- grant_type,
- scope

```
curl -X POST http://localhost:12345/oauth/token --user jlcapp:jlc123 -H  
"content-type: application/x-www-form-urlencoded" -d "code= mrYiWW  
&grant_type=authorization_code&redirect_uri=http://localhost:5000/callback&scope=read_profile"
```

The token endpoint will verify all the parameters in the request, ensuring the code hasn't expired and that the client ID and secret match. If everything is ok, it will generate an access token and return it in the response!

```
{  
  "access_token": "0cbc284e-6ae0-4ce8-a4b3-c7eafd4d98e0",  
  "token_type": "bearer",  
  "refresh_token": "907368bf-c6bb-46f0-a365-ad7b235b9191",  
  "expires_in": 119,  
  "scope": "read_profile"  
}
```

Step 3: Now access the BookPrice by providing the retrieved access

You can now Invoke your Resources or REST API with the Access Token received above.

```
curl -X GET http://localhost:12345/myapi/bookPrice/105 -H "authorization:  
Bearer 0cbc284e-6ae0-4ce8-a4b3-c7eafd4d98e0"
```

Now you can get the Access Token again with Refresh Token when your Access token expires.

Step A)

A) Endpoint - /oauth/ token

B) Query Parameters

- clientId,
- clientSecret
- grant_type=refresh_token
- refresh_token= -----



```
curl -X POST http://localhost:12345/oauth/token --user jlcapp:jlc123 -H  
"content-type: application/x-www-form-urlencoded" -d  
"grant_type=refresh_token&refresh_token=907368bf-c6bb-46f0-a365-  
ad7b235b9191&scope=read_profile"
```

```
{  
  "access_token":"06fb5475-af0c-4b07-a191-ce5f3b74e69c",  
  "token_type":"bearer",  
  "refresh_token":"907368bf-c6bb-46f0-a365-ad7b235b9191",  
  "expires_in":119,  
  "scope":"read_profile"  
}
```

Step B: Now access the BookPrice by providing the retrieved access

```
curl -X GET http://localhost:12345/myapi/bookPrice/105 -H "authorization:  
Bearer 06fb5475-af0c-4b07-a191-ce5f3b74e69c"
```

B)Using Refresh Token with Authorization Code

Running Steps:

Step1: Send POST Request to Authorization Server to get the Access Token

A) Endpoint - /oauth/token

B) Query Parameters

- ClientID , Client Scret , grant_type=password
- RO username , RO password , scope

```
curl -X POST http://localhost:12345/oauth/token --user jlcapp:jlc123 -H  
"content-type: application/x-www-form-urlencoded" -d  
"grant_type=password&username=srinivas&password=srinivas&scope=read  
_profile"
```

Above POST request will send the Access Token in the Response as follows

```
{  
  "access_token":"5d72bc03-1875-42c9-978e-57ec6aac1ee9",  
  "token_type":"bearer",  
  "refresh_token":"907368bf-c6bb-46f0-a365-ad7b235b9191",  
  "expires_in":119,  
  "scope":"read_profile"  
}
```

Step 2: Now access the BookPrice by providing the retrieved access

You can now invoke your Resources or REST API with the Access Token received above.

```
curl -X GET http://localhost:12345/myapi/bookPrice/105 -H "authorization: Bearer 5d72bc03-1875-42c9-978e-57ec6aac1ee9"
```

Now you can get the Access Token again with Refresh Token when your Access token expires.**Step A)**

A) Endpoint - /oauth/ token

B) Query Parameters

- clientId,
- clientSecret
- grant_type=refresh_token
- refresh_token= -----

```
curl -X POST http://localhost:12345/oauth/token --user jlcapp:jlc123 -H "content-type: application/x-www-form-urlencoded" -d "grant_type=refresh_token&refresh_token=907368bf-c6bb-46f0-a365-ad7b235b9191&scope=read_profile"
```

```
{
  "access_token": "06fb5475-af0c-4b07-a191-ce5f3b74e69c",
  "token_type": "bearer",
  "refresh_token": "907368bf-c6bb-46f0-a365-ad7b235b9191",
  "expires_in": 119,
  "scope": "read_profile"
}
```

Step B: Now access the BookPrice by providing the retrieved access

```
curl -X GET http://localhost:12345/myapi/bookPrice/105 -H "authorization: Bearer 06fb5475-af0c-4b07-a191-ce5f3b74e69c"
```



refresh-token-type-server – Files Required

1. MyApplication.java	Same as Above
2. BookPriceInfo.java	Same as Above
3. BookPriceController.java	Same as Above
4. MyResourceServer.java	Same as Above
5. MyAuthorizationServer.java	Updated in password Type
6. application.properties	Updated in password Type
7. pom.xml	Updated in password Type

5. MyAuthorizationServer.java

```
package com.coursecube.oauth2;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.oauth2.config.annotation.configurers.*;
import org.springframework.security.oauth2.config.annotation.web.configuration.*;
import org.springframework.security.oauth2.config.annotation.web.configurers.*;

/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */

@Configuration
@EnableAuthorizationServer
public class MyAuthorizationServer extends AuthorizationServerConfigurerAdapter {

    @Autowired
    private AuthenticationManager authenticationManager;

    @Override
    public void configure(AuthorizationServerEndpointsConfigurer endpoints) throws Exception
    {
        endpoints.authenticationManager(authenticationManager);
    }

    @Override
    public void configure(ClientDetailsServiceConfigurer clients) throws Exception {
```



```
clients.inMemory()  
.withClient("jlcapp")  
.secret("jlc123")  
.redirectUri("http://localhost:5000/callback")  
.authorizedGrantTypes("authorization_code", "password", "refresh_token")  
.accessTokenValiditySeconds(120)  
.scopes("read_profile", "read_contacts");  
  
}  
}
```

6. application.properties

```
spring.application.name=refresh-token-type-server  
  
server.port=12345  
  
security.user.name=srinivas  
security.user.password=srinivas
```

7. pom.xml

```
<project ...>  
<modelVersion>4.0.0</modelVersion>  
<groupId>com.coursecube.oauth2</groupId>  
<artifactId>refresh-token-type-server</artifactId>  
<version>1.0</version>  
<packaging>jar</packaging>  
  
<name>refresh-token-type-server</name>  
<url>https://www.coursecube.com</url>  
<description>This is refresh-token-type-server by Srinivas Dande</description>  
  
<properties>  
<maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>  
<java.version>1.8</java.version>  
</properties>  
<parent>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-parent</artifactId>  
<version>1.5.2.RELEASE</version>  
<relativePath />  
</parent>
```



```
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

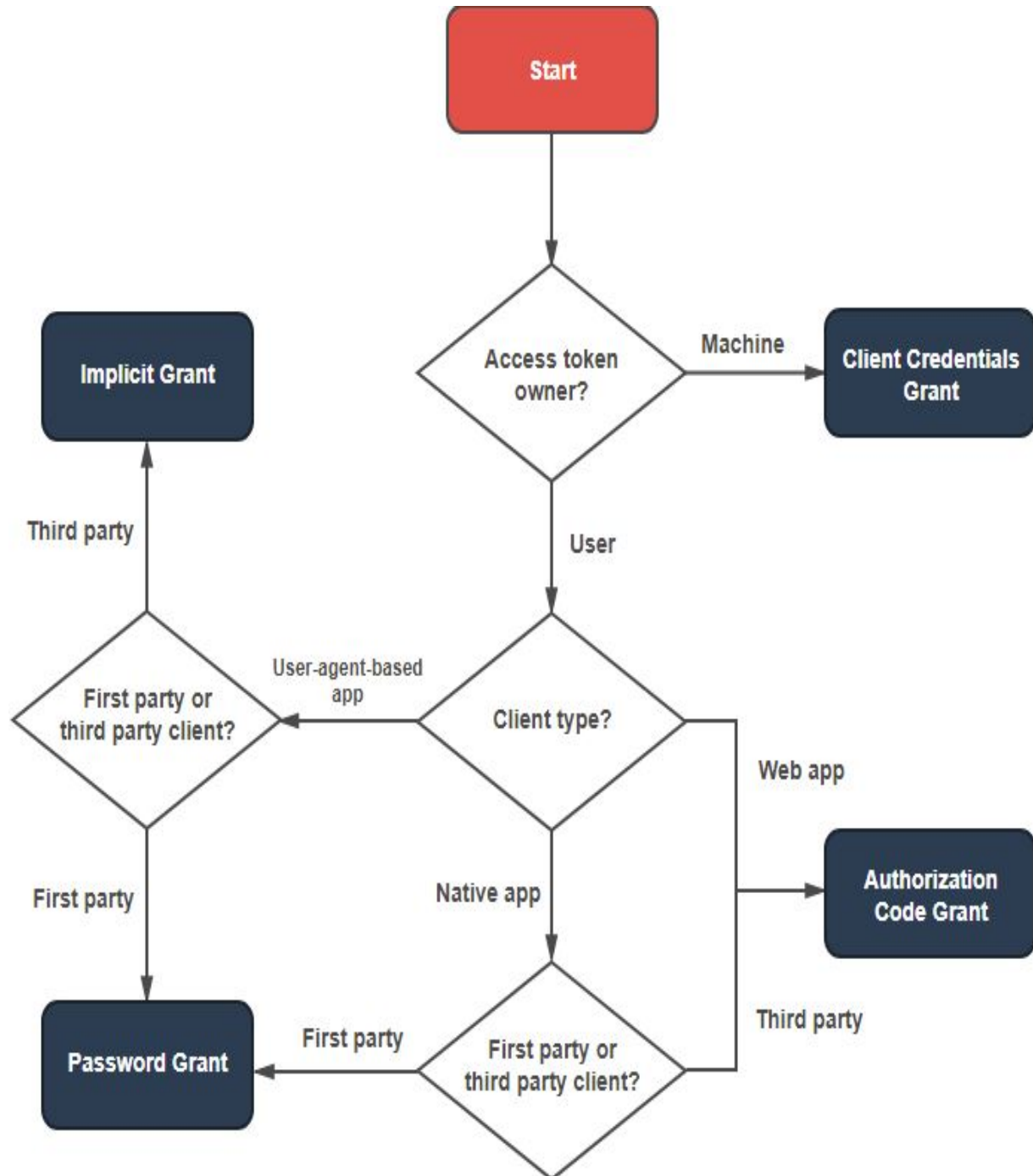
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.security.oauth</groupId>
<artifactId>spring-security-oauth2</artifactId>
</dependency>

</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



Choosing Required Grant Type





Access Token Owner?

- An access token represents a permission granted to a client to access some protected resources.
- If you are authorizing a machine to access resources and you don't require the permission of a user to access said resources you should implement the client credentials grant.
- If you require the permission of a user to access resources you need to determine the client type.

First party or third party client?

- A first party client is a client that you trust enough to handle the end user's authorization credentials. For example Spotify's iPhone app is owned and developed by Spotify so therefore they implicitly trust it.
- A third party client is a client that you don't trust.

Client Type?

- Depending on whether or not the client is capable of keeping a secret will depend on which grant the client should use.
- If the client is a web application that has a server side component then you should implement the authorization code grant.
- If the client is a web application that has runs entirely on the front end (e.g. a single page web application) you should implement the password grant for a first party clients and the implicit grant for a third party clients
- If the client is a native application such as a mobile app you should implement the password grant.



OAuth2 Client Applications

We have developed 4 Server Applications for 4 Grant Types as follows.

- authorization-code-type-server
- implicit-type-server
- ro-password-type-server
- client-credentials-type-server

In the Above Applications, We have written the following for **BookPriceMS**

- BookPriceController.java
- BookPriceInfo.java

Now add the following for **UserRatingMS**.

- UserRatingController.java
- UserRatingInfo.java

We will call both **BookPriceMS** API and **UserRatingMS** API from **OAuth2 Client Applications**

UserRatingController.java

```
package com.coursecube.oauth2;

import java.util.ArrayList;
import java.util.List;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@RestController
@RequestMapping("/myapi")
public class UserRatingController {

    @GetMapping("/userRating/{userId}")
    public List<UserRatingInfo> getUserRating(@PathVariable String userId) {
```



```
System.out.println("---BookPriceController---getBookPrice()----: " );
```

```
UserRatingInfo userRatingInfo = new UserRatingInfo(userId,101,4.5,"Best One");
List<UserRatingInfo> mylist=new ArrayList<>();
mylist.add(userRatingInfo);
mylist.add(userRatingInfo);
mylist.add(userRatingInfo);
mylist.add(userRatingInfo);
mylist.add(userRatingInfo);
return mylist;
}
}
```

UserRatingInfo.java

```
package com.coursecube.oauth2;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
public class UserRatingInfo {

    private String userId;
    private Integer bookId;
    private double rating;
    private String review;

    public UserRatingInfo() {}
    public UserRatingInfo(String userId, Integer bookId, double rating, String review) {
        super();
        this.userId = userId;
        this.bookId = bookId;
        this.rating = rating;
        this.review = review;
    }
    //Setters and Getters
    //toString()

}
```

OAuth2 Client for Authorization Code Grant Type

Working Steps:

- 1) Copy authorization-code-type-server as **authorization-code-type-client**
- 2) Update **pom.xml**
- 3) Update **application.properties**
- 4) Create the Source Folder called **src/main/webapp**
- 5) Create **WEB-INF** folder under **src/main/webapp**
- 6) Create **myjsps** folder under **WEB-INF**
- 7) Write the following jsp's under **myjsps**
 - a. index.jsp
 - b. bookInfo.jsp
 - c. ratingInfo.jsp
- 8) Write the following classes under **com.coursecube.oauth2**
 - a. MyClientApplication.java
 - b. BookStoreWebController.java
 - c. BookPriceInfo.java
 - d. UserRatingInfo.java
 - e. OAuth2ClientConfiguration.java
 - f. SecurityConfiguration.java
 - g. AppConfiguration.java
- 9) Run the App

authorization-code-type-client – Files Required

1. MyClientApplication.java	2. index.jsp
3. bookInfo.jsp	4. ratingInfo.jsp
5. BookStoreWebController.java	6. BookPriceInfo.java
7. UserRatingInfo.java	8. OAuth2ClientConfiguration.java
9. SecurityConfiguration.java	10. AppConfiguration.java
11. application.properties	12. pom.xml



1. MyClientApplication.java

```
package com.coursecube.oauth2;

import javax.servlet.ServletContext;
import javax.servlet.ServletException;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.boot.web.servlet.ServletContextInitializer;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@SpringBootApplication
public class MyClientApplication implements ServletContextInitializer {
    public static void main(String args[]) {
        SpringApplication.run(MyClientApplication.class, args);
    }
    @Override
    public void onStartup(ServletContext servletContext) throws ServletException {
        servletContext.getSessionCookieConfig().setName("client-session");
    }
}
```

2. index.jsp

```
<!DOCTYPE html>
<html>
<head>    <title>JLCBookStoreWeb - AN OAuth2 Client</title>
<style>

```



3.bookInfo.jsp

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
```

```
<html>
<head>
<title>JLC Bookstore</title>
<style>
div,table{
font-size:25px;
}
</style>
</head>
<body>

<div align="center">
<h2>JLCBookStoreWeb - An OAuth2 Client</h2>
<h3>Book Price Information</h3>
<table class="table">
<tr>
<td><b>BookId</b></td>
<td>${mybookPriceInfo.bookId}</td>
</tr>
<tr>
<td><b>price</b></td>
<td>${mybookPriceInfo.price}</td>
</tr>
<tr>
<td><b>offer</b></td>
<td>${mybookPriceInfo.offer}</td>
</tr>
</table>
</div>
<div align="center">
<h3>
<a href="/">Go to Home</a>
</h3>
</div>

</body>
</html>
```



4.ratingInfo.jsp

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
```

```
<html>
<head>
<title>JLC Bookstore</title>
<style>
div,table{
font-size:25px;
}
</style>
</head>
<body>

<div align="center">
<h2>JLCBookStoreWeb - An OAuth2 Client</h2>
<h3>Book Rating Information</h3>
<table class="table">
<tr>
<td><b>UserId</b></td>
<td><b>BookID</b></td>
<td><b>Rating</b></td>
<td><b>review</b></td>
</tr>
<c:forEach var="myraingInfo" items="${myRatingList}">
<tr>
<td>${myraingInfo.userId}</td>
<td>${myraingInfo.bookId}</td>
<td>${myraingInfo.rating}</td>
<td>${myraingInfo.review}</td>
</tr>
</c:forEach>
</table>
</div>
<div align="center">
<h3>
<a href="/">Go to Home</a>
</h3>
</div>

</body>
</html>
```



5. BookStoreWebController.java

```
package com.coursecube.oauth2;

import java.util.List;
import javax.servlet.http.HttpSession;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.oauth2.client.OAuth2RestTemplate;
import org.springframework.security.oauth2.common.OAuth2AccessToken;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.client.HttpClientErrorException;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@Controller
public class BookStoreWebController {

    @Autowired
    private OAuth2RestTemplate restTemplate;

    @GetMapping("/")
    public String home() {
        System.out.println("----home----");
        /*
        OAuth2ClientContext clientCtx=restTemplate.getOAuth2ClientContext();
        OAuth2AccessToken mytoken= clientCtx.getAccessToken();
        OAuth2ProtectedResourceDetails resourceDetails = restTemplate.getResource();
        */
        return "index";
    }

    @GetMapping("/callback")
    public String callback() {
        System.out.println("----callback----");
        return "forward:/mybookPriceInfo";
    }
}
```



```
@GetMapping("/mybookPriceInfo")
public String getBookPriceInfo(HttpSession session) {
    System.out.println("----getBookPriceInfo----");

    String endpoint = "http://localhost:12345/myapi/bookPrice/105";
    try {
        System.out.println("Before BookPriceInfo : ");
        OAuth2AccessToken mytoken1=restTemplate.getAccessToken();
        System.out.println(mytoken1.getValue());
        System.out.println(mytoken1.getTokenType());
        System.out.println(mytoken1.getExpiresIn());
        System.out.println(mytoken1.getScope());
        System.out.println(mytoken1.getRefreshToken());

        BookPriceInfo bookPriceInfo = restTemplate.getForObject(endpoint,
BookPriceInfo.class);

        System.out.println("After BookPriceInfo : ");
        OAuth2AccessToken mytoken2=restTemplate.getAccessToken();
        System.out.println(mytoken2.getValue());
        System.out.println(mytoken2.getTokenType());
        System.out.println(mytoken2.getExpiresIn());
        System.out.println(mytoken2.getScope());
        System.out.println(mytoken2.getRefreshToken());

        session.setAttribute("mybookPriceInfo", bookPriceInfo);
    } catch (HttpClientErrorException e) {
        throw new RuntimeException("Some Problem in Connecting BookPriceMS");
    }

    return "bookInfo";
}

@GetMapping("/mybookRatingInfo")
public String getRatingInfo(HttpSession session) {
    System.out.println("----getRatingInfo----");

    String endpoint = "http://localhost:12345/myapi/userRating/U-999";
    try {
        System.out.println("Before UserRatingInfo : ");
        OAuth2AccessToken mytoken1=restTemplate.getAccessToken();
        System.out.println(mytoken1.getValue());
```



```
System.out.println(mytoken1.getTokenType());
System.out.println(mytoken1.getExpiresIn());
System.out.println(mytoken1.getScope());
System.out.println(mytoken1.getRefreshToken());
```

```
List<UserRatingInfo> mylist = restTemplate.getForObject(endpoint, List.class);
```

```
System.out.println("After UserRatingInfo : ");
OAuth2AccessToken mytoken2=restTemplate.getAccessToken();
System.out.println(mytoken2.getValue());
System.out.println(mytoken2.getTokenType());
System.out.println(mytoken2.getExpiresIn());
System.out.println(mytoken2.getScope());
System.out.println(mytoken2.getRefreshToken());
session.setAttribute("myRatingList", mylist);
} catch (HttpClientErrorException e) {
throw new RuntimeException("Some Problem in Connecting UserRatingMS");
}
return "ratingInfo";
}
}
```

6. BookPriceInfo.java

```
package com.coursecube.oauth2;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
public class BookPriceInfo {
    private Integer bookId;
    private double price;
    private double offer;

    public BookPriceInfo() {}
    public BookPriceInfo(Integer bookId, double price, double offer) {
        super();
        this.bookId = bookId;
        this.price = price;
        this.offer = offer;
    }
    //Setters and Getters
}
```



7. UserRatingInfo.java

```
package com.coursecube.oauth2;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
public class UserRatingInfo {

    private String userId;
    private Integer bookId;
    private double rating;
    private String review;

    public UserRatingInfo() {}
    public UserRatingInfo(String userId, Integer bookId, double rating, String review) {
        super();
        this.userId = userId;
        this.bookId = bookId;
        this.rating = rating;
        this.review = review;
    }
    //Setters and Getters
}
```

8. OAuth2ClientConfiguration.java

```
package com.coursecube.oauth2;

import java.util.Arrays;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.client.OAuth2ClientContext;
import org.springframework.security.oauth2.client.OAuth2RestTemplate;
import org.springframework.security.oauth2.client.resource.*;
import org.springframework.security.oauth2.client.token.*;
import org.springframework.security.oauth2.client.token.grant.code.*;
import org.springframework.security.oauth2.common.*;
import org.springframework.security.oauth2.config.annotation.web.configuration.*;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
```



@Configuration

@EnableOAuth2Client

```
public class OAuth2ClientConfiguration {
```

@Autowired

```
private OAuth2ClientContext oauth2ClientContext;
```

@Bean

```
public OAuth2ProtectedResourceDetails resourceDetails() {  
AuthorizationCodeResourceDetails resourceDetails = new  
AuthorizationCodeResourceDetails();
```

```
resourceDetails.setId("oauth2server"); // 1  
resourceDetails.setTokenName("oauth_token"); // 2  
resourceDetails.setClientId("jlcapp"); // 3  
resourceDetails.setClientSecret("jlc123"); // 4  
resourceDetails.setGrantType("authorization_code"); // 5
```

```
resourceDetails.setUserAuthorizationUri("http://localhost:12345/oauth/authorize"); //6  
resourceDetails.setAccessTokenUri("http://localhost:12345/oauth/token"); //7  
resourceDetails.setPreEstablishedRedirectUri(("http://localhost:5000/callback")); //8  
resourceDetails.setScope(Arrays.asList("read_profile")); //9  
resourceDetails.setUseCurrentUri(false); //10  
resourceDetails.setClientAuthenticationScheme(AuthenticationScheme.header); //11
```

```
return resourceDetails;  
}
```

@Bean

```
public OAuth2RestTemplate oauth2RestTemplate() {
```

```
OAuth2RestTemplate template = new OAuth2RestTemplate(  
resourceDetails(), oauth2ClientContext);
```

```
AccessTokenProviderChain provider = new AccessTokenProviderChain(  
Arrays.asList(new AuthorizationCodeAccessTokenProvider()));
```

```
template.setAccessTokenProvider(provider);
```

```
return template;  
}  
}
```



9. SecurityConfiguration.java

```
package com.coursecube.oauth2;

import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.authentication.builders.*;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.*;
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@Configuration
@EnableWebSecurity
public class SecurityConfiguration extends WebSecurityConfigurerAdapter {

    @Override
    public void configure(AuthenticationManagerBuilder auth) throws Exception {
        auth.inMemoryAuthentication()
            .withUser("hello123").password("hello123").roles("USER")
            .and().withUser("hai123").password("hai123").roles("USER");
    }

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http.authorizeRequests()
            .antMatchers("/**").permitAll()
            .anyRequest().authenticated()
            .and()
            .formLogin().and().logout().permitAll();
    }
}
```

10. AppConfigConfiguration.java

```
package com.coursecube.oauth2;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.EnableWebMvc;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurerAdapter;
import org.springframework.web.servlet.view.InternalResourceViewResolver;
import org.springframework.web.servlet.view.JstlView;
/*
 * @Author : Srinivas Dande

```



```
* @company : CourseCube
* @website : www.coursecube.com
* */
@Configuration
@EnableWebMvc
public class AppConfig extends WebMvcConfigurerAdapter {

    @Bean
    public InternalResourceViewResolver viewResolver() {
        InternalResourceViewResolver viewResolver = new InternalResourceViewResolver();
        viewResolver.setViewClass(JstlView.class);
        viewResolver.setPrefix("/WEB-INF/jsp/");
        viewResolver.setSuffix(".jsp");
        return viewResolver;
    }
}
```

11. application.properties

```
spring.application.name=authorization-code-client
server.port=5000
spring.http.converters.preferred-json-mapper=jackson
```

12. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.coursecube.oauth2</groupId>
<artifactId>authorization-code-type-client</artifactId>
<version>1.0</version>
<packaging>jar</packaging>
<name>authorization-code-type-client</name>
<url>https://www.coursecube.com</url>
<description>This is authorization-code-type-client developed by Srinivas</description>

<properties>
<maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>
<java.version>1.8</java.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>1.5.2.RELEASE</version>
<relativePath />
</parent>
```



```
<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.security.oauth</groupId>
<artifactId>spring-security-oauth2</artifactId>
</dependency>

<dependency>
<groupId>org.apache.tomcat.embed</groupId>
<artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<dependency>
<groupId>jstl</groupId>
<artifactId>jstl</artifactId>
<version>1.2</version>
</dependency>

<dependency>
<groupId>>taglibs</groupId>
<artifactId>standard</artifactId>
<version>1.1.2</version>
</dependency>
</dependencies>
<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



OAuth2 Client for RO Password Grant Type

Working Steps:

- 1) Copy **authorization-code-type-client** as **ro-password-type-client**
- 2) Update **pom.xml**
- 3) Update **application.properties**
- 4) Update **BookStoreWebController.java**
- 5) Update **OAuth2ClientConfiguration.java**
- 6) Run the App

ro-password -type-client – Files Required

1. MyClientApplication.java	Same as Above
2. index.jsp	Same as Above
3. bookInfo.jsp	Same as Above
4. ratingInfo.jsp	Same as Above
5. BookStoreWebController.java	Updated in this Lab
6. BookPriceInfo.java	Same as Above
7. UserRatingInfo.java	Same as Above
8. OAuth2ClientConfiguration.java	Updated in this Lab
9. SecurityConfiguration.java	Same as Above
10.AppConfiguration.java	Same as Above
11.application.properties	Updated in this Lab
12.pom.xml	Updated in this Lab

5. BookStoreWebController.java

```
package com.coursecube.oauth2;

import java.util.List;
import javax.servlet.http.HttpSession;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.oauth2.client.OAuth2RestTemplate;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.client.HttpClientErrorException;
```



```
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */

@Controller
public class BookStoreWebController {

    @Autowired
    private OAuth2RestTemplate restTemplate;

    @GetMapping("/")
    public String home() {
        System.out.println("----home----");
        return "index";
    }

    @GetMapping("/mybookPriceInfo")
    public String getBookPriceInfo(HttpSession session) {
        System.out.println("----getBookPriceInfo----");

        String endpoint = "http://localhost:12345/myapi/bookPrice/105";
        try {
            System.out.println("Before Call : ");
            System.out.println(restTemplate.getAccessToken());

            BookPriceInfo bookPriceInfo = restTemplate.getForObject(endpoint,
                BookPriceInfo.class);

            System.out.println("After Call : ");
            System.out.println(restTemplate.getAccessToken());

            session.setAttribute("mybookPriceInfo", bookPriceInfo);
        } catch (HttpClientErrorException e) {
            throw new RuntimeException("Some Problem in Connecting BookPriceMS");
        }
        return "bookInfo";
    }

    @GetMapping("/mybookRatingInfo")
    public String getRatingInfo(HttpSession session) {
        System.out.println("----getRatingInfo----");
    }
}
```



```
String endpoint = "http://localhost:12345/myapi/userRating/U-999";
try {
    System.out.println("Before Call : ");
    System.out.println(restTemplate.getAccessToken());

    List<UserRatingInfo> mylist = restTemplate.getForObject(endpoint, List.class);

    System.out.println("After Call : ");
    System.out.println(restTemplate.getAccessToken());

    session.setAttribute("myRatingList", mylist);
} catch (HttpClientErrorException e) {
    throw new RuntimeException("Some Problem in Connecting UserRatingMS");
}

return "ratingInfo";
}
}
```

8. OAuth2ClientConfiguration.java

```
package com.coursecube.oauth2;

import java.util.Arrays;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.client.OAuth2ClientContext;
import org.springframework.security.oauth2.client.OAuth2RestTemplate;
import org.springframework.security.oauth2.client.resource.*;
import org.springframework.security.oauth2.client.token.AccessTokenProviderChain;
import org.springframework.security.oauth2.client.token.grant.password.*;
import org.springframework.security.oauth2.common.AuthenticationScheme;
import org.springframework.security.oauth2.config.annotation.web.configuration.*;

/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
```



@Configuration

@EnableOAuth2Client

public class **OAuth2ClientConfiguration** {

@Autowired

private OAuth2ClientContext oauth2ClientContext;

@Bean

public OAuth2ProtectedResourceDetails resourceDetails() {
ResourceOwnerPasswordResourceDetails resourceDetails =
new ResourceOwnerPasswordResourceDetails();

resourceDetails.setId("**oauth2server**"); // 1
resourceDetails.setTokenName("**oauth_token**"); // 2
resourceDetails.setClientId("**jl capp**"); // 3
resourceDetails.setClientSecret("**jl c123**"); // 4
resourceDetails.setGrantType("**password**"); // 5

resourceDetails.setUsername("**srinivas**"); //6
resourceDetails.setPassword("**srinivas**"); //7

resourceDetails.setAccessTokenUri("**http://localhost:12345/oauth/token**"); //8
resourceDetails.setScope(Arrays.asList("**read_profile**")); //9
resourceDetails.setClientAuthenticationScheme(**AuthenticationScheme.header**); //10

return **resourceDetails**;
}

@Bean

public OAuth2RestTemplate oauth2RestTemplate() {
OAuth2RestTemplate template = new OAuth2RestTemplate(
resourceDetails(), oauth2ClientContext);

AccessTokenProviderChain provider = new AccessTokenProviderChain(
Arrays.asList(new ResourceOwnerPasswordAccessTokenProvider()));

template.setAccessTokenProvider(provider);

return **template**;
}
}



11. application.properties

spring.application.name=ro-password-type-client

server.port=5000

spring.http.converters.preferred-json-mapper=jackson

12. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.coursecube.oauth2</groupId>
<artifactId>ro-password-type-client</artifactId>
<version>1.0</version>
<packaging>jar</packaging>

<name>ro-password-type-client</name>
<url>https://www.coursecube.com</url>
<description>This is ro-password-type-clientt by Srinivas Dande</description>

<properties>
<maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>
<java.version>1.8</java.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>1.5.2.RELEASE</version>
<relativePath />
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.security.oauth</groupId>
<artifactId>spring-security-oauth2</artifactId>
</dependency>

<dependency>
<groupId>org.apache.tomcat.embed</groupId>
<artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<dependency>
<groupId>jstl</groupId>
<artifactId>jstl</artifactId>
<version>1.2</version>
</dependency>

<dependency>
<groupId>>taglibs</groupId>
<artifactId>standard</artifactId>
<version>1.1.2</version>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



OAuth2 Client for Client Credentials Grant Type

Working Steps:

- 1) Copy **ro-password-type-client** as **client-credentials-type-client**
- 2) Update **pom.xml**
- 3) Update **application.properties**
- 4) Update **OAuth2ClientConfiguration.java**
- 5) Run the App

client-credentials-type-client – Files Required

1. MyClientApplication.java	Same as Above
2. index.jsp	Same as Above
3. bookInfo.jsp	Same as Above
4. ratingInfo.jsp	Same as Above
5. BookStoreWebController.java	Same as Above
6. BookPriceInfo.java	Same as Above
7. UserRatingInfo.java	Same as Above
8. OAuth2ClientConfiguration.java	Updated in this Lab
9. SecurityConfiguration.java	Same as Above
10.AppConfiguration.java	Same as Above
11. application.properties	Updated in this Lab
12. pom.xml	Updated in this Lab

8. OAuth2ClientConfiguration.java

```
package com.coursecube.oauth2;

import java.util.Arrays;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.client.OAuth2ClientContext;
import org.springframework.security.oauth2.client.OAuth2RestTemplate;
import org.springframework.security.oauth2.client.resource.*;
import org.springframework.security.oauth2.client.token.*;
import org.springframework.security.oauth2.client.token.grant.client.*;
import org.springframework.security.oauth2.common.*;
import org.springframework.security.oauth2.config.annotation.web.configuration.*;
```



```
/*
 * @Author : Srinivas Dande
 * @company : CourseCube
 * @website : www.coursecube.com
 */
@Configuration
@EnableOAuth2Client
public class OAuth2ClientConfiguration {

    @Autowired
    private OAuth2ClientContext oauth2ClientContext;

    @Bean
    public OAuth2ProtectedResourceDetails resourceDetails() {
        ClientCredentialsResourceDetails resourceDetails =
        new ClientCredentialsResourceDetails();

        resourceDetails.setClientId("oauth2server"); // 1
        resourceDetails.setTokenName("oauth_token"); // 2
        resourceDetails.setClientId("jlcapp"); // 3
        resourceDetails.setClientSecret("jlc123"); // 4
        resourceDetails.setGrantType("client_credentials"); // 5

        resourceDetails.setAccessTokenUri("http://localhost:12345/oauth/token"); //6
        resourceDetails.setScope(Arrays.asList("read_profile")); //7
        resourceDetails.setClientAuthenticationScheme(AuthenticationScheme.header); //9

        return resourceDetails;
    }

    @Bean
    public OAuth2RestTemplate oauth2RestTemplate() {

        OAuth2RestTemplate template = new OAuth2RestTemplate(
        resourceDetails(), oauth2ClientContext);

        AccessTokenProviderChain provider = new AccessTokenProviderChain(
        Arrays.asList(new ClientCredentialsAccessTokenProvider()));

        template.setAccessTokenProvider(provider);

        return template;
    }
}
```



11. application.properties

spring.application.name=client-credentials-type-client

server.port=5000

spring.http.converters.preferred-json-mapper=jackson

12. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.coursecube.oauth2</groupId>
<artifactId>client-credentials-type-client</artifactId>
<version>1.0</version>
<packaging>jar</packaging>

<name>client-credentials-type-client</name>
<url>https://www.coursecube.com</url>
<description>This is client-credentials-type-client by Srinivas Dande</description>

<properties>
<maven-jar-plugin.version>3.1.1</maven-jar-plugin.version>
<java.version>1.8</java.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>1.5.2.RELEASE</version>
<relativePath />
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.security.oauth</groupId>
<artifactId>spring-security-oauth2</artifactId>
</dependency>

<dependency>
<groupId>org.apache.tomcat.embed</groupId>
<artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<dependency>
<groupId>jstl</groupId>
<artifactId>jstl</artifactId>
<version>1.2</version>
</dependency>

<dependency>
<groupId>>taglibs</groupId>
<artifactId>standard</artifactId>
<version>1.1.2</version>
</dependency>

</dependencies>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```